CSA4704 - DEEP LEARNING
Lab Experiments

Name: Harini. R
Register no: 192324108

Experiment 16: LSTM and RNN Performance Comparison

Problem Statement: A food-delivery platform wants to forecast daily order volume for the next week. Long-term weekly and seasonal patterns matter. Two years of daily order counts with weekly and festival-season patterns (or an airline passenger dataset). The objective is to compare SimpleRNN and LSTM on a long-dependency forecasting task. Use TensorFlow, Keras, NumPy, Pandas, Matplotlib.

Tasks:

1. Prepare sequences with a 30-day look-back window.
2. Train a SimpleRNN and an LSTM with the same number of units.
3. Compare training loss, validation loss and training time.
4. Compute MAE, RMSE and MAPE on the test set.
5. Plot both forecasts against actual values; tabulate the results with Pandas.

Formulas:

$$h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1} + b_h)$$

Forget gate:

$$f_t = \sigma(W_f [h_{t-1}, x_t] + b_f)$$

Input gate:

$$i_t = \sigma(W_i [h_{t-1}, x_t] + b_i)$$

Candidate cell state:

$$\widetilde{C}_t = \tanh(W_C [h_{t-1}, x_t] + b_C)$$

Cell state:

$$C_t = f_t * C_{t-1} + i_t * \widetilde{C}_t$$

Output gate:

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

Hidden state:

$$h_t = o_t * \tanh(C_t)$$

Mean Absolute Error:

$$MAE = \frac{1}{n}\sum_{i=1}^{n} |y_i - \hat{y}_i|$$

Mean Squared Error:

$$MSE = \frac{1}{n}\sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

Root Mean Squared Error:

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n} (y_i - \hat{y}_i)^2}$$

Mean Absolute Percentage Error:

$$MAPE = \frac{100}{n}\sum_{i=1}^{n} \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

Code:

```
# Install only if TensorFlow is not already available
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
```

```
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, LSTM, Dense

np.random.seed(42)
tf.random.set_seed(42)

# 1. Generate two years of daily order counts
days = 730
t = np.arange(days)

orders = (
    500
    + 80 * np.sin(2 * np.pi * t / 7)       # weekly pattern
    + 100 * np.sin(2 * np.pi * t / 365)    # yearly seasonality
    + 0.15 * t                             # gradual growth
    + np.random.normal(0, 25, days)        # random noise
)
orders = np.maximum(orders, 0)

df = pd.DataFrame({
    "Date": pd.date_range("2024-01-01", periods=days, freq="D"),
    "Orders": orders
}).set_index("Date")

plt.figure(figsize=(12, 4))
plt.plot(df.index, df["Orders"])
plt.title("Daily Food Delivery Orders")
plt.xlabel("Date")
plt.ylabel("Order Count")
plt.show()

# 2. Chronological train/validation/test split
values = df["Orders"].values.reshape(-1, 1)
train_end = int(len(values) * 0.70)
val_end = int(len(values) * 0.85)

# Fit scaler on training data only
scaler = MinMaxScaler()
scaler.fit(values[:train_end])
scaled = scaler.transform(values)

LOOKBACK = 30
```

```
def make_sequences(data, start, end, lookback=30):
    X, y = [], []
    for i in range(max(start, lookback), end):
        X.append(data[i-lookback:i, 0])
        y.append(data[i, 0])
    return np.array(X), np.array(y)

X_train, y_train = make_sequences(scaled, LOOKBACK, train_end, LOOKBACK)
X_val, y_val = make_sequences(scaled, train_end, val_end, LOOKBACK)
X_test, y_test = make_sequences(scaled, val_end, len(scaled), LOOKBACK)

# RNNs expect [samples, time steps, features]
X_train = X_train[..., np.newaxis]
X_val = X_val[..., np.newaxis]
X_test = X_test[..., np.newaxis]

# 3. Build comparable models with the same number of recurrent units
UNITS = 32

def build_model(cell_type):
    model = Sequential()
    model.add(tf.keras.Input(shape=(LOOKBACK, 1)))

    if cell_type == "SimpleRNN":
        model.add(SimpleRNN(UNITS))
    elif cell_type == "LSTM":
        model.add(LSTM(UNITS))

    model.add(Dense(1))
    model.compile(optimizer="adam", loss="mse")
    return model

results = []
histories = {}
predictions = {}

for cell_type in ["SimpleRNN", "LSTM"]:
    tf.keras.backend.clear_session()
    model = build_model(cell_type)

    start_time = time.time()
    history = model.fit(
        X_train, y_train,
```

```
        validation_data=(X_val, y_val),
        epochs=20,
        batch_size=32,
        shuffle=False,
        verbose=0,
        callbacks=[
            tf.keras.callbacks.EarlyStopping(
                monitor="val_loss", patience=4,
                restore_best_weights=True
            )
        ]
    )
    training_time = time.time() - start_time

    pred_scaled = model.predict(X_test, verbose=0)
    pred = scaler.inverse_transform(pred_scaled).ravel()
    actual = scaler.inverse_transform(y_test.reshape(-1, 1)).ravel()

    mae = mean_absolute_error(actual, pred)
    rmse = np.sqrt(mean_squared_error(actual, pred))
    # Avoid division by zero in MAPE
    mape = np.mean(np.abs((actual - pred) / np.maximum(np.abs(actual), 1e-8))) * 100

    results.append({
        "Model": cell_type,
        "Units": UNITS,
        "Epochs Run": len(history.history["loss"]),
        "Training Time (seconds)": round(training_time, 2),
        "Test MAE": round(mae, 2),
        "Test RMSE": round(rmse, 2),
        "Test MAPE (%)": round(mape, 2),
        "Parameters": model.count_params()
    })

    histories[cell_type] = history.history
    predictions[cell_type] = pred

# 4. Plot training and validation loss
plt.figure(figsize=(10, 5))
for name, history in histories.items():
    plt.plot(history["loss"], label=f"{name} training")
    plt.plot(history["val_loss"], linestyle="--", label=f"{name} validation")
plt.title("Training and Validation Loss")
plt.xlabel("Epoch")
```

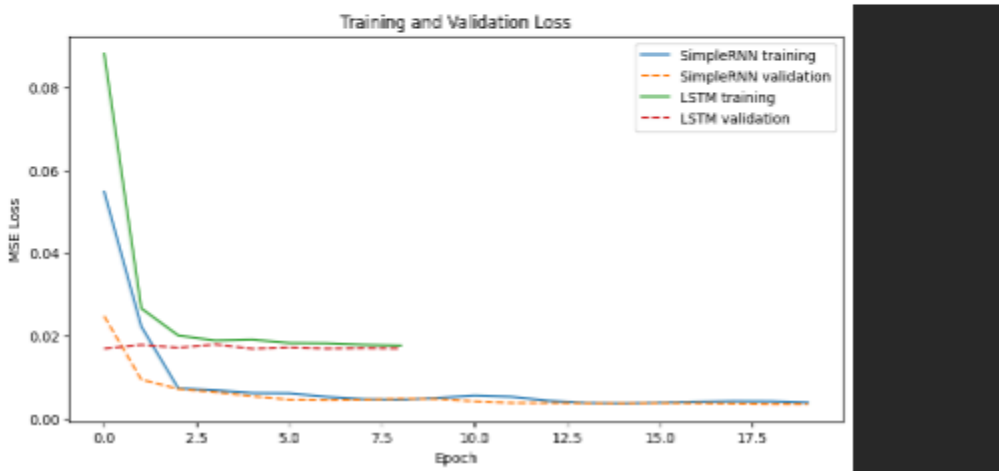
```
plt.ylabel("MSE Loss")
plt.legend()
plt.show()

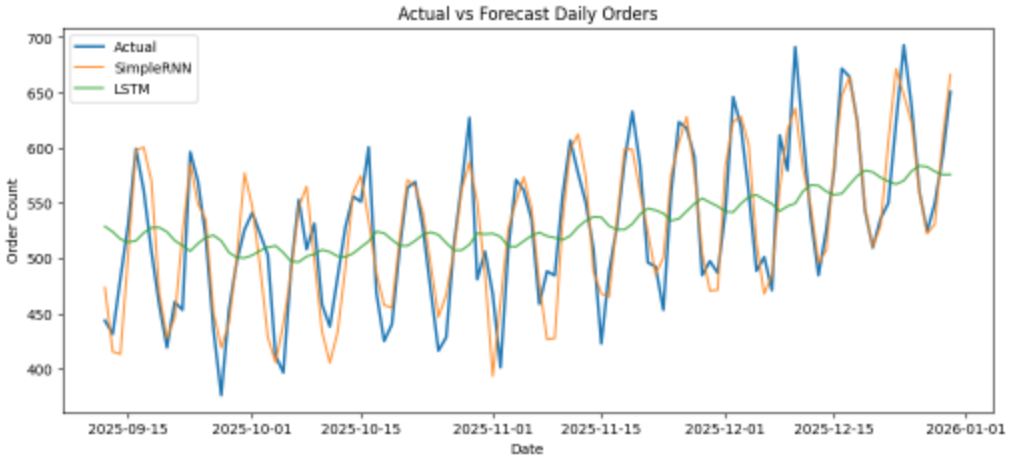
# 5. Plot forecasts against actual test orders
plt.figure(figsize=(12, 5))
plt.plot(df.index[val_end:], actual, label="Actual", linewidth=2)
for name, pred in predictions.items():
    plt.plot(df.index[val_end:], pred, label=name, alpha=0.8)
plt.title("Actual vs Forecast Daily Orders")
plt.xlabel("Date")
plt.ylabel("Order Count")
plt.legend()
plt.show()

# 6. Tabulate performance
results_df = pd.DataFrame(results)
display(results_df)
```

Output:







	Model	Units	Epochs Run	Training Time (seconds)	Test MAE	Test RMSE	Test MAPE (%)	Parameters
0	SimpleRNN	32	20	13.43	25.23	31.71	4.95	1121
1	LSTM	32	9	10.02	54.17	63.63	10.55	4385

Experiment 17: RNN Gradient Analysis and Gradient Clipping
Problem Statement: A text-generation model at a content start-up produces NaN losses midway through training because of exploding gradients. A character-level text corpus (for example, a public-domain novel) trained with a deliberately high learning rate. The objective is to detect exploding gradients and stabilise training with gradient clipping. Use TensorFlow, Keras, NumPy, Matplotlib.
Tasks:
1. Train an RNN without clipping and log the global gradient norm each step.
2. Identify the steps where the gradient spikes or loss becomes NaN.
3. Retrain with clipnorm and with clipvalue at different thresholds.
4. Plot gradient norm and loss curves for each setting.
5. Recommend a clipping threshold.

Formulas:

1. Backpropagation through time

$$\frac{\partial L}{\partial W} = \sum_{t=1}^{T} \frac{\partial L_t}{\partial W}$$

2. Gradient norm

$$||g||_2 = \sqrt{\sum_{i=1}^{n} g_i^2}$$

3. Gradient clipping

$$g_{clipped} = g \cdot \min\left(1, \frac{threshold}{||g||_2}\right)$$

4. Weight update

$$W_{new} = W_{old} - \eta g_{clipped}$$

Where:

- g = gradient
- η = learning rate
- $threshold$ = maximum allowed gradient norm

Code:

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from collections import Counter

np.random.seed(42)
tf.random.set_seed(42)

# 1. Small character-level corpus
text = (
    "the little robot learns from examples. "
    "the robot reads words and predicts the next character. "
    "language models learn patterns from sequences. "
) * 150

chars = sorted(set(text))
char_to_id = {ch: i for i, ch in enumerate(chars)}
id_to_char = {i: ch for ch, i in char_to_id.items()}
encoded = np.array([char_to_id[ch] for ch in text], dtype=np.int32)

SEQ_LEN = 40
X = np.array([encoded[i:i+SEQ_LEN] for i in range(len(encoded)-SEQ_LEN)])
y = np.array([encoded[i+SEQ_LEN] for i in range(len(encoded)-SEQ_LEN)])

# Use chronological split
```

```
split = int(0.9 * len(X))
X_train, y_train = X[:split], y[:split]
X_val, y_val = X[split:], y[split:]

VOCAB_SIZE = len(chars)
BATCH_SIZE = 64

def build_rnn():
    model = tf.keras.Sequential([
        tf.keras.Input(shape=(SEQ_LEN,), dtype=tf.int32),
        tf.keras.layers.Embedding(VOCAB_SIZE, 32),
        tf.keras.layers.SimpleRNN(64),
        tf.keras.layers.Dense(VOCAB_SIZE)
    ])
    return model

# 2. Custom training loop: monitor gradient norm at every step
def train_with_clipping(setting, epochs=5, learning_rate=1.0):
    tf.keras.backend.clear_session()
    model = build_rnn()
    optimizer = tf.keras.optimizers.SGD(learning_rate=learning_rate)
    loss_fn = tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True)

    grad_norms, losses = [], []
    stopped_early = False

    train_ds = tf.data.Dataset.from_tensor_slices(
        (X_train, y_train)
    ).shuffle(2000, seed=42).batch(BATCH_SIZE)

    for epoch in range(epochs):
        for xb, yb in train_ds:
            with tf.GradientTape() as tape:
                logits = model(xb, training=True)
                loss = loss_fn(yb, logits)

            grads = tape.gradient(loss, model.trainable_variables)
            valid_grads = [g for g in grads if g is not None]

            # Measure the original, unclipped global norm
            norm = tf.linalg.global_norm(valid_grads)
            norm_value = float(norm.numpy())

            if not np.isfinite(float(loss.numpy())) or not np.isfinite(norm_value):
```

```
            stopped_early = True
            break

        grad_norms.append(norm_value)
        losses.append(float(loss.numpy()))

        # 3. Apply clipping, if selected
        if setting == "clipnorm":
            grads, _ = tf.clip_by_global_norm(grads, 1.0)
        elif setting == "clipvalue":
            grads = [
                None if g is None else tf.clip_by_value(g, -0.5, 0.5)
                for g in grads
            ]

        optimizer.apply_gradients(
            (g, v) for g, v in zip(grads, model.trainable_variables)
            if g is not None
        )

    if stopped_early:
        break

return {
    "setting": setting,
    "norms": grad_norms,
    "losses": losses,
    "stopped_early": stopped_early
}

# 4. Compare no clipping, global-norm clipping and value clipping
runs = []
for setting in ["none", "clipnorm", "clipvalue"]:
    result = train_with_clipping(setting)
    runs.append(result)
    print(
        setting,
        "| steps:", len(result["losses"]),
        "| stopped early:", result["stopped_early"]
    )

# 5. Plot gradient norms and losses
fig, axes = plt.subplots(2, 1, figsize=(12, 8))
```

```
for result in runs:
    axes[0].plot(result["norms"], label=result["setting"])
    axes[1].plot(result["losses"], label=result["setting"])

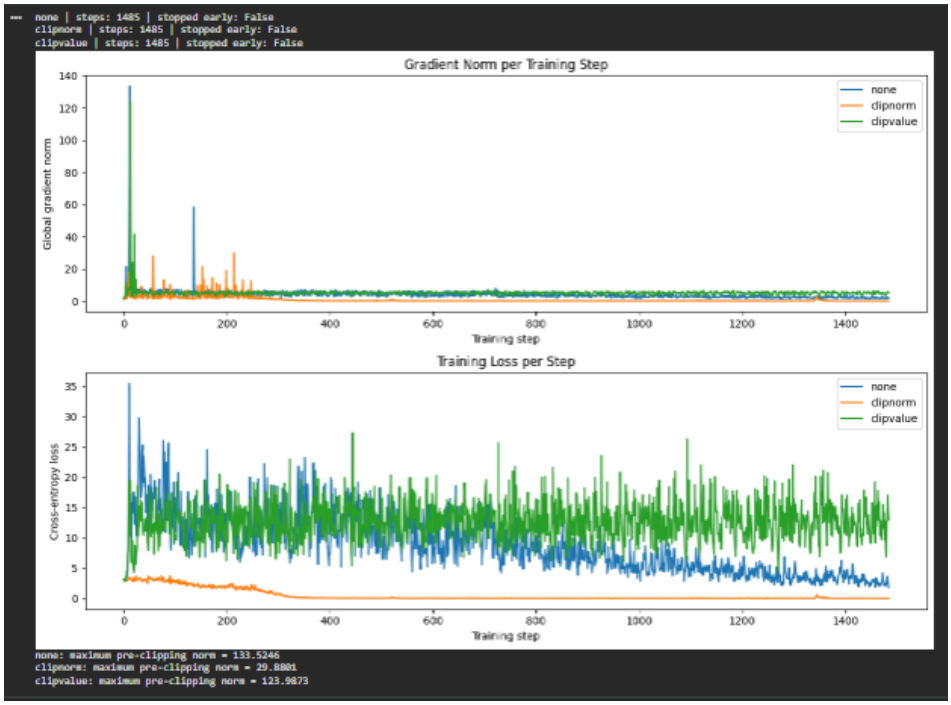
axes[0].set_title("Gradient Norm per Training Step")
axes[0].set_ylabel("Global gradient norm")
axes[0].set_xlabel("Training step")
axes[0].legend()

axes[1].set_title("Training Loss per Step")
axes[1].set_ylabel("Cross-entropy loss")
axes[1].set_xlabel("Training step")
axes[1].legend()

plt.tight_layout()
plt.show()

# Report the largest observed gradient norm for each setting
for result in runs:
    if result["norms"]:
        print(
            f"{result['setting']}: maximum pre-clipping norm = "
            f"{max(result['norms']):.4f}"
        )
```

Output:



Experiment 18: RNN, LSTM and GRU Hyperparameter Experimentation
Problem Statement: A product team classifies customer reviews as positive or negative. It must pick the best recurrent model and its settings within a fixed compute budget. The IMDB movie-review sentiment dataset (or product reviews from an e-commerce site). The objective is to tune RNN, LSTM and GRU models automatically with KerasTuner. Use TensorFlow, Keras, NumPy, KerasTuner, Matplotlib.
Tasks:
1. Tokenise and pad the reviews; add an embedding layer.
2. Define a model-builder function that chooses the cell type (RNN, LSTM, GRU).
3. Tune units, layers, dropout, learning rate and embedding size with KerasTuner.
4. Report the best configuration for each cell type.
5. Compare accuracy, parameter count and training time.

Formulas:

1. Encoder

$$h = f(W_e x + b_e)$$

2. Decoder

$$\hat{x} = g(W_d h + b_d)$$

3. Reconstruction loss (MSE)

$$L = \frac{1}{n}\sum_{i=1}^{n}(x_i - \hat{x}_i)^2$$

4. Total objective

$$\min_{\theta,\phi} L(x, \hat{x})$$

Where θ and ϕ represent encoder and decoder parameters.

Code:

```
# Run once in Google Colab if needed:
%pip -q install keras-tuner
```

```
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
import keras_tuner as kt

from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras import layers, Sequential

np.random.seed(42)
tf.random.set_seed(42)

# 1. Load and prepare reviews
VOCAB_SIZE = 10000
MAX_LEN = 200

(x_train, y_train), (x_test, y_test) = imdb.load_data(
    num_words=VOCAB_SIZE
)

# Use a subset to keep the search manageable
x_train, y_train = x_train[:8000], y_train[:8000]
x_test, y_test = x_test[:2000], y_test[:2000]

x_train = pad_sequences(x_train, maxlen=MAX_LEN, padding="post", truncating="post")
x_test = pad_sequences(x_test, maxlen=MAX_LEN, padding="post", truncating="post")

# Separate validation data from the end of the training subset
x_val, y_val = x_train[6000:], y_train[6000:]
x_train_small, y_train_small = x_train[:6000], y_train[:6000]

# 2. Model builder: cell type, units, layers, dropout,
#    learning rate and embedding size are tunable.
def build_model(hp):
    cell_type = hp.Choice("cell_type", ["SimpleRNN", "LSTM", "GRU"])
    embedding_dim = hp.Choice("embedding_dim", [32, 64])
    units = hp.Choice("units", [32, 64])
    num_layers = hp.Int("num_layers", min_value=1, max_value=2)
    dropout_rate = hp.Float("dropout", 0.2, 0.5, step=0.1)
    learning_rate = hp.Choice("learning_rate", [1e-3, 5e-4])
```

```
    model = Sequential()
    model.add(tf.keras.Input(shape=(MAX_LEN,)))
    model.add(layers.Embedding(VOCAB_SIZE, embedding_dim))

    for i in range(num_layers):
        return_sequences = (i < num_layers - 1)

        if cell_type == "SimpleRNN":
            model.add(layers.SimpleRNN(
                units, return_sequences=return_sequences
            ))
        elif cell_type == "LSTM":
            model.add(layers.LSTM(
                units, return_sequences=return_sequences
            ))
        else:
            model.add(layers.GRU(
                units, return_sequences=return_sequences
            ))

        model.add(layers.Dropout(dropout_rate))

    model.add(layers.Dense(1, activation="sigmoid"))
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=learning_rate),
        loss="binary_crossentropy",
        metrics=["accuracy"]
    )
    return model

# 3. Tune the models. The cell type is a search dimension.
tuner = kt.RandomSearch(
    build_model,
    objective="val_accuracy",
    max_trials=6,
    executions_per_trial=1,
    directory="imdb_tuning",
    project_name="rnn_lstm_gru",
    overwrite=True,
    seed=42
)

start = time.time()
tuner.search(
```

```
    x_train_small, y_train_small,
    validation_data=(x_val, y_val),
    epochs=4,
    batch_size=64,
    callbacks=[
        tf.keras.callbacks.EarlyStopping(
            monitor="val_loss", patience=1, restore_best_weights=True
        )
    ],
    verbose=1
)
search_time = time.time() - start

# 4. Report the best configuration found for each cell type
all_trials = []
for trial in tuner.oracle.trials.values():
    if trial.score is not None:
        hp_values = trial.hyperparameters.values.copy()
        hp_values["Validation Accuracy"] = trial.score
        all_trials.append(hp_values)

trials_df = pd.DataFrame(all_trials)
display(trials_df.sort_values("Validation Accuracy", ascending=False))

# Because cell_type is a search dimension, the search budget may not
# test every cell type. Train the best available trial for each type
# to make a fair, explicit comparison.
comparison = []

for cell_type in ["SimpleRNN", "LSTM", "GRU"]:
    matching = [
        trial for trial in tuner.oracle.trials.values()
        if trial.score is not None
        and trial.hyperparameters.get("cell_type") == cell_type
    ]

    if not matching:
        print(f"No completed trial for {cell_type}. Increase max_trials to test it.")
        continue

    best_trial = max(matching, key=lambda tr: tr.score)
    hp = best_trial.hyperparameters

    # Build a fixed model using the best trial's settings
```

```
    model = build_model(hp)

    start = time.time()
    history = model.fit(
        x_train_small, y_train_small,
        validation_data=(x_val, y_val),
        epochs=5,
        batch_size=64,
        callbacks=[
            tf.keras.callbacks.EarlyStopping(
                monitor="val_loss", patience=1, restore_best_weights=True
            )
        ],
        verbose=0
    )
    training_time = time.time() - start

    test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=0)

    comparison.append({
        "Cell Type": cell_type,
        "Best Units": hp.get("units"),
        "Layers": hp.get("num_layers"),
        "Dropout": hp.get("dropout"),
        "Embedding Size": hp.get("embedding_dim"),
        "Learning Rate": hp.get("learning_rate"),
        "Test Accuracy": round(test_accuracy, 4),
        "Test Loss": round(test_loss, 4),
        "Parameters": model.count_params(),
        "Training Time (seconds)": round(training_time, 2)
    })

comparison_df = pd.DataFrame(comparison)
display(comparison_df)

# 5. Compare test accuracy
if not comparison_df.empty:
    plt.figure(figsize=(8, 4))
    plt.bar(comparison_df["Cell Type"], comparison_df["Test Accuracy"])
    plt.title("Test Accuracy: SimpleRNN vs LSTM vs GRU")
    plt.ylabel("Accuracy")
    plt.ylim(0, 1)
    plt.show()
```

print(f"Total hyperparameter-search time: {search_time:.2f} seconds")

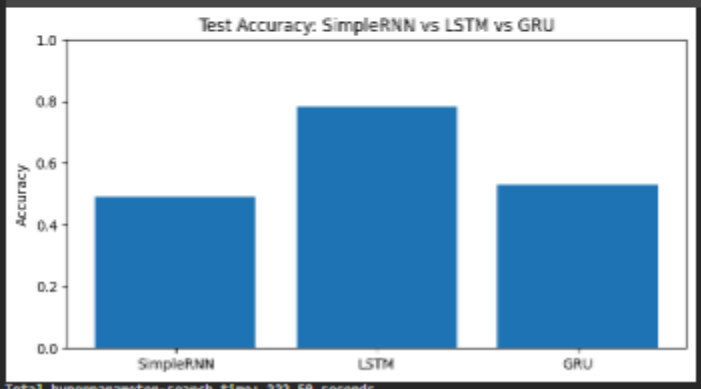
Output:

```
Trial 6 Complete [00h 00m 21s]
val_accuracy: 0.5164999961853027

Best val_accuracy So Far: 0.6539999842643738
Total elapsed time: 00h 03m 42s
```

	cell_type	embedding_dim	units	num_layers	dropout	learning_rate	Validation Accuracy
1	LSTM	32	32	1	0.4	0.0005	0.6540
0	LSTM	32	32	1	0.4	0.0010	0.5435
2	GRU	32	64	1	0.3	0.0010	0.5230
3	SimpleRNN	32	64	1	0.3	0.0010	0.5170
5	SimpleRNN	32	64	1	0.3	0.0005	0.5165
4	SimpleRNN	32	64	2	0.4	0.0010	0.5000

	Cell Type	Best Units	Layers	Dropout	Embedding Size	Learning Rate	Test Accuracy	Test Loss	Parameters	Training Time (seconds)
0	SimpleRNN	64	1	0.3	32	0.0010	0.4925	0.6946	326273	22.31
1	LSTM	32	1	0.4	32	0.0005	0.7835	0.5326	328353	50.81
2	GRU	64	1	0.3	32	0.0010	0.5295	0.6931	338881	54.98




```
Total hyperparameter-search time: 222.50 seconds
```

CO4 – Linear Factor Models and Probabilistic Graphical Models
Experiment 19: PCA and Explained Variance Analysis
Problem Statement: A face-recognition attendance system stores thousands of pixels per face, making matching slow. The team needs fewer features without losing identity information. The Olivetti faces dataset (400 images, 64×64 pixels) or the handwritten digits dataset. The objective is to reduce dimensionality with PCA and choose components using explained variance. Use NumPy, Scikit-learn, Matplotlib.
Tasks:
1. Implement PCA from scratch using the covariance matrix and eigen-decomposition.
2. Verify the result against Scikit-learn's PCA.
3. Plot the scree plot and cumulative explained variance.
4. Find the number of components that retain 95% of variance.
5. Visualise the top eigenfaces and reconstructed faces with 10, 50 and 100 components.

Formulas:

1. Noisy input

$$\tilde{x} = x + \epsilon$$

Where ϵ is added noise.

2. Encoder

$$h = f(W_e \tilde{x} + b_e)$$

3. Decoder

$$\hat{x} = g(W_d h + b_d)$$

4. Reconstruction loss

$$L = \frac{1}{n} \sum_{i=1}^{n} (x_i - \hat{x}_i)^2$$

The model receives noisy data but tries to reconstruct the original clean data.

Code:

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import fetch_olivetti_faces
from sklearn.decomposition import PCA
from sklearn.metrics import mean_squared_error

np.random.seed(42)

# 1. Load face images
faces_data = fetch_olivetti_faces(shuffle=True, random_state=42)
X = faces_data.data
images = faces_data.images

print("Dataset shape:", X.shape)
print("Image shape:", images.shape)

# Display sample faces
```

```
fig, axes = plt.subplots(1, 5, figsize=(12, 3))
for i, ax in enumerate(axes):
    ax.imshow(images[i], cmap="gray")
    ax.axis("off")
    ax.set_title(f"Face {i+1}")
plt.suptitle("Sample Olivetti Faces")
plt.show()

# 2. PCA from scratch using the covariance matrix's dual form.
# Directly forming a 4096 x 4096 covariance matrix is expensive.
# The dual covariance matrix is only 400 x 400 and has the
# same non-zero eigenvalues.
mean_face = X.mean(axis=0)
X_centered = X - mean_face
n_samples = X.shape[0]

dual_cov = (X_centered @ X_centered.T) / (n_samples - 1)

eigenvalues, eigenvectors_dual = np.linalg.eigh(dual_cov)

# Sort eigenvalues from largest to smallest
order = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[order]
eigenvectors_dual = eigenvectors_dual[:, order]

# Convert dual eigenvectors into eigenvectors in pixel space
components = X_centered.T @ eigenvectors_dual

# Normalize each component to unit length
norms = np.linalg.norm(components, axis=0)
components = components / np.maximum(norms, 1e-12)

# Remove tiny negative eigenvalues caused by numerical precision
eigenvalues = np.maximum(eigenvalues, 0)

explained_variance_ratio = eigenvalues / np.sum(eigenvalues)
cumulative_variance = np.cumsum(explained_variance_ratio)

# 3. Verify with Scikit-learn PCA
pca = PCA(svd_solver="full")
pca.fit(X)

print("\nFirst 10 eigenvalues (from scratch):")
print(np.round(eigenvalues[:10], 6))
```

```
print("\nFirst 10 eigenvalues (Scikit-learn):")
print(np.round(pca.explained_variance_[:10], 6))

print("\nMaximum difference in first 10 eigenvalues:",
      np.max(np.abs(eigenvalues[:10] - pca.explained_variance_[:10])))

# Compare subspaces by comparing the absolute cosine similarity
# of the first five corresponding principal directions.
similarities = np.abs(np.sum(components[:, :5] * pca.components_[:5], axis=0))
print("Absolute similarity of first 5 components:")
print(np.round(similarities, 4))

# 4. Scree plot and cumulative explained variance
plt.figure(figsize=(10, 4))
plt.plot(np.arange(1, len(eigenvalues)+1), explained_variance_ratio * 100)
plt.title("Scree Plot")
plt.xlabel("Principal Component")
plt.ylabel("Explained Variance (%)")
plt.grid(True)
plt.show()

plt.figure(figsize=(10, 4))
plt.plot(np.arange(1, len(cumulative_variance)+1),
         cumulative_variance * 100)
plt.axhline(95, linestyle="--", label="95% variance")
plt.title("Cumulative Explained Variance")
plt.xlabel("Number of Components")
plt.ylabel("Cumulative Variance (%)")
plt.legend()
plt.grid(True)
plt.show()

n_95 = np.searchsorted(cumulative_variance, 0.95) + 1
print("Components needed for at least 95% variance:", n_95)

# 5. Visualize the top eigenfaces
fig, axes = plt.subplots(2, 5, figsize=(12, 5))
for i, ax in enumerate(axes.flat):
    ax.imshow(pca.components_[i].reshape(64, 64), cmap="gray")
    ax.set_title(f"PC {i+1}")
    ax.axis("off")
plt.suptitle("Top 10 Eigenfaces")
plt.tight_layout()
```

```
plt.show()

# 6. Reconstruct an example face with 10, 50 and 100 components
face_index = 0
original = X[face_index].reshape(64, 64)

fig, axes = plt.subplots(1, 4, figsize=(12, 3))
axes[0].imshow(original, cmap="gray")
axes[0].set_title("Original")
axes[0].axis("off")

for ax, k in zip(axes[1:], [10, 50, 100]):
    pca_k = PCA(n_components=k, svd_solver="full")
    pca_k.fit(X)
    z = pca_k.transform(X[face_index:face_index+1])
    reconstructed = pca_k.inverse_transform(z)[0].reshape(64, 64)

    ax.imshow(reconstructed, cmap="gray")
    ax.set_title(f"{k} components")
    ax.axis("off")

plt.suptitle("Face Reconstruction with PCA")
plt.tight_layout()
plt.show()

# Reconstruction error across component counts
component_counts = [5, 10, 20, 50, 100, 150, 200]
errors = []

for k in component_counts:
    pca_k = PCA(n_components=k, svd_solver="full")
    Z = pca_k.fit_transform(X)
    X_reconstructed = pca_k.inverse_transform(Z)
    errors.append(mean_squared_error(X, X_reconstructed))

plt.figure(figsize=(8, 4))
plt.plot(component_counts, errors, marker="o")
plt.title("Reconstruction Error vs Number of Components")
plt.xlabel("Number of Components")
plt.ylabel("Mean Squared Error")
plt.grid(True)
plt.show()

print("Reconstruction errors:")
```

```
for k, err in zip(component_counts, errors):
    print(f"{k:3d} components: MSE = {err:.6f}")
```

Output:

```
Dataset shape: (400, 4096)
Image shape: (400, 64, 64)
```

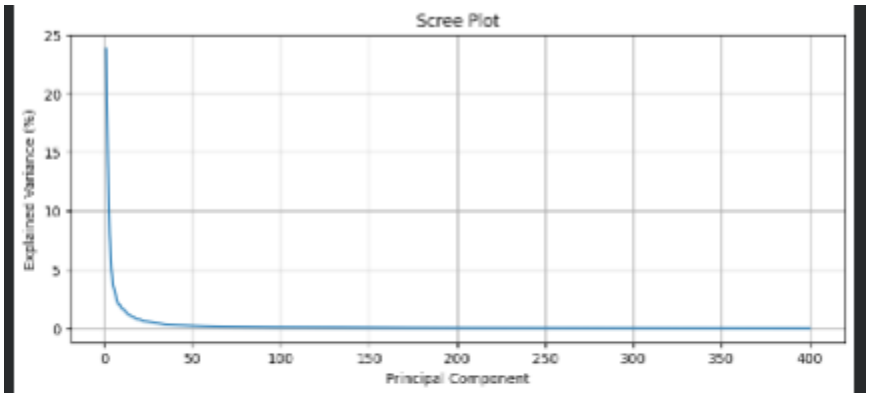


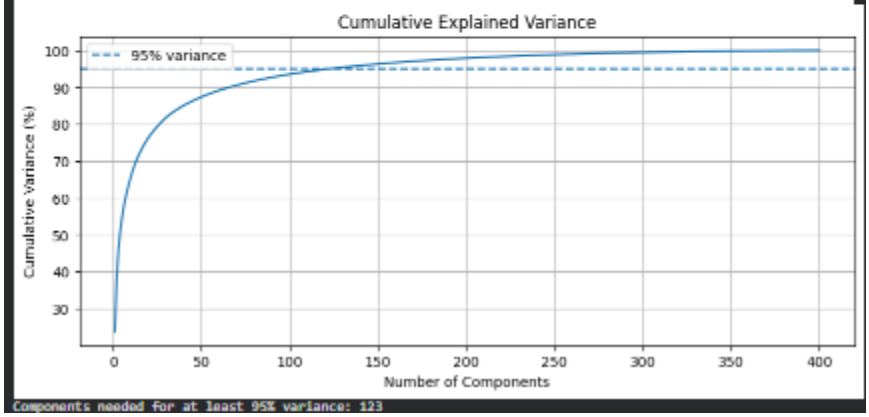

```
First 10 eigenvalues (from scratch):
[18.848176 11.071762  6.304614  3.954584  2.856042  2.49771   1.920863
  1.611159  1.549222  1.322951]

First 10 eigenvalues (Scikit-learn):
[18.848168 11.071753  6.304612  3.954582  2.856037  2.497712  1.920863
  1.611159  1.549222  1.322951]

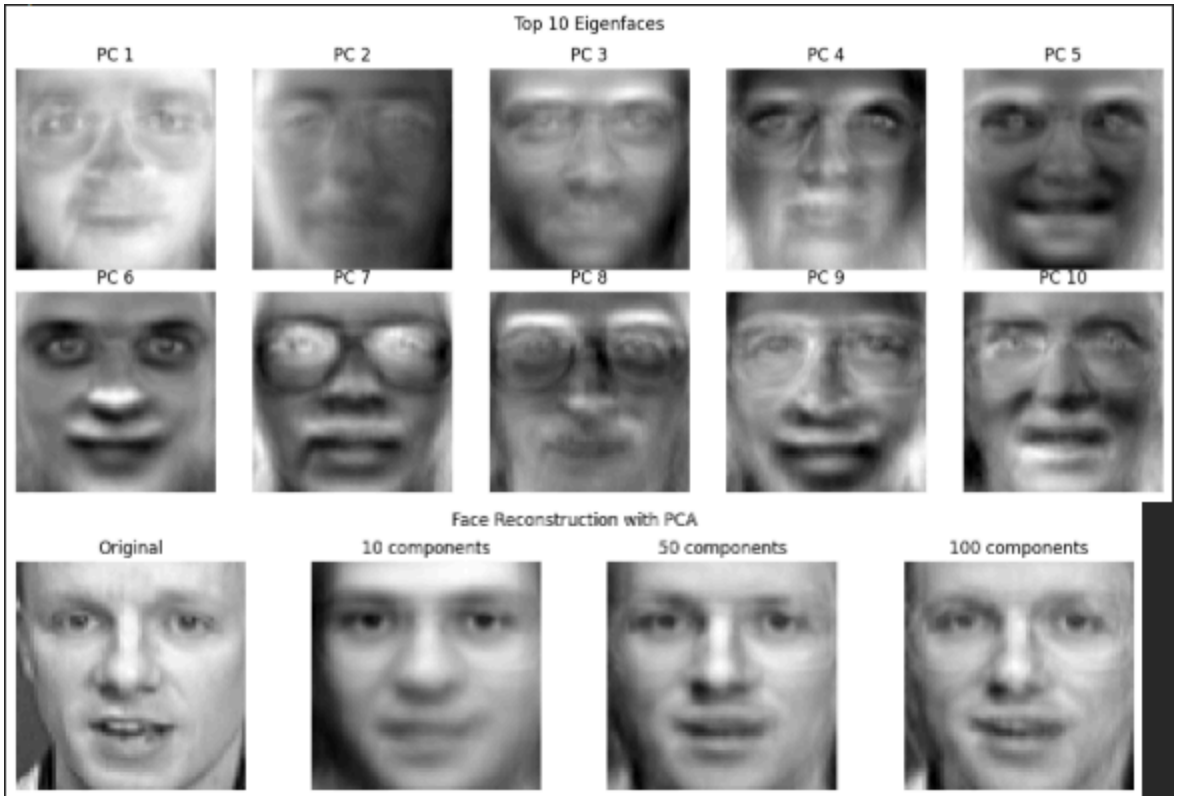
Maximum difference in first 10 eigenvalues:
9.536743e-06

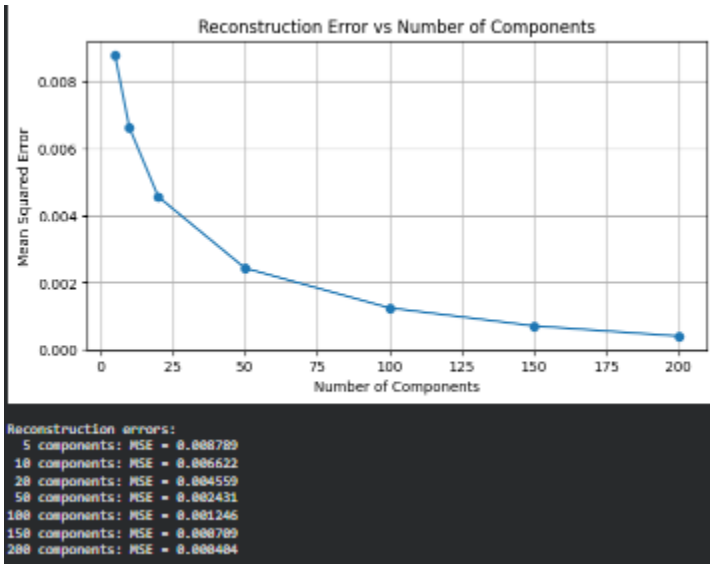
Absolute similarity of first 5 components:
```






```
Components needed for at least 95% variance: 123
```






```
Reconstruction errors:
  5 components: MSE = 0.008789
 10 components: MSE = 0.006622
 20 components: MSE = 0.004559
 50 components: MSE = 0.002431
100 components: MSE = 0.001246
150 components: MSE = 0.000789
200 components: MSE = 0.000484
```

Experiment 20: Probabilistic PCA and Reconstruction Analysis

Problem Statement: A sensor network in a smart factory produces readings with random noise and occasional missing values. Engineers want a model that accounts for noise explicitly. Multivariate sensor readings (temperature, vibration, pressure, current and so on) with added Gaussian noise. The objective is to apply probabilistic PCA and compare its reconstruction with standard PCA. Use NumPy, Scikit-learn, Matplotlib.

Tasks:

1. Explain the PPCA generative model $x = Wz + \mu + \varepsilon$.

2. Fit PPCA (Scikit-learn PCA with its noise-variance estimate, or an EM implementation in NumPy).
3. Estimate the noise variance σ2.
4. Reconstruct the data and compute reconstruction error against component count.
5. Generate new synthetic sensor samples from the fitted model.

Formulas:

1. Mean and variance

$$\mu = W_{\mu} h + b_{\mu}$$

$$\log(\sigma^2) = W_{\sigma} h + b_{\sigma}$$

2. Reparameterization trick

$$z = \mu + \sigma \odot \epsilon$$

$$\epsilon \sim N(0, I)$$

3. KL divergence

$$D_{KL}(q(z|x)||p(z)) = -\frac{1}{2}\sum_{j=1}^{d}(1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2)$$

4. VAE loss

$$L_{VAE} = L_{reconstruction} + D_{KL}(q(z|x)||p(z))$$

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.decomposition import PCA
from sklearn.impute import SimpleImputer
from sklearn.metrics import mean_squared_error

np.random.seed(42)

# 1. Generate synthetic multivariate sensor data
n_samples = 1000

# Two underlying latent factors
Z = np.random.normal(size=(n_samples, 2))

# Factor loadings: 2 latent factors -> 5 sensor readings
W_true = np.array([
    [2.0, 0.3],
    [1.5, 0.5],
    [0.2, 2.0],
    [0.4, 1.5],
    [1.0, 1.0]
])

mu_true = np.array([25, 30, 100, 10, 50])

# Add Gaussian sensor noise
noise_std = 0.5
noise = np.random.normal(0, noise_std, size=(n_samples, 5))

X_complete = Z @ W_true.T + mu_true + noise

columns = ["Temperature", "Vibration", "Pressure", "Current", "Humidity"]
df = pd.DataFrame(X_complete, columns=columns)

# 2. Introduce occasional missing values
missing_mask = np.random.random(df.shape) < 0.03
df_missing = df.mask(missing_mask)

print("Missing values per sensor:")
print(df_missing.isna().sum())
```

```
# PCA cannot directly accept NaN values, so impute missing values.
# Fit the imputer on the available dataset.
imputer = SimpleImputer(strategy="mean")
X = imputer.fit_transform(df_missing)

print("\nPrepared data shape:", X.shape)

# Plot sensor distributions
df_missing.hist(figsize=(10, 6), bins=25)
plt.suptitle("Sensor Reading Distributions")
plt.tight_layout()
plt.show()

# 3. Fit PPCA-style model with 2 principal components
K = 2
ppca = PCA(n_components=K)
Z_est = ppca.fit_transform(X)
X_reconstructed = ppca.inverse_transform(Z_est)

# In sklearn PCA, noise_variance_ estimates the average variance
# in the dimensions not retained by the model.
sigma2_est = ppca.noise_variance_

print("Estimated PPCA noise variance (sigma^2):", round(sigma2_est, 6))
print("Explained variance by each retained component:")
print(np.round(ppca.explained_variance_, 4))
print("Explained variance ratio:")
print(np.round(ppca.explained_variance_ratio_, 4))
print("Total retained variance (%):",
      round(100 * ppca.explained_variance_ratio_.sum(), 2))

# 4. Compare reconstruction error across component counts
component_counts = [1, 2, 3, 4]

errors = []
models = {}

for k in component_counts:
    model = PCA(n_components=k)
    transformed = model.fit_transform(X)
    reconstructed = model.inverse_transform(transformed)

    mse = mean_squared_error(X, reconstructed)
```

```
    errors.append(mse)
    models[k] = model

results_df = pd.DataFrame({
    "Components": component_counts,
    "Explained Variance (%)": [
        models[k].explained_variance_ratio_.sum() * 100
        for k in component_counts
    ],
    "Reconstruction MSE": errors
})

display(results_df)

plt.figure(figsize=(8, 4))
plt.plot(component_counts, errors, marker="o")
plt.title("Reconstruction Error vs Number of Components")
plt.xlabel("Number of Principal Components")
plt.ylabel("Mean Squared Error")
plt.xticks(component_counts)
plt.grid(True)
plt.show()

# 5. Compare original and reconstructed readings for two sensors
plt.figure(figsize=(10, 5))
sample_ids = np.arange(100)

plt.plot(sample_ids, X[:100, 0], label="Actual Temperature")
plt.plot(sample_ids, X_reconstructed[:100, 0],
         label="Reconstructed Temperature", linestyle="--")

plt.title("Actual vs PPCA-Style Reconstruction")
plt.xlabel("Sample")
plt.ylabel("Temperature Reading")
plt.legend()
plt.show()

# 6. Generate new samples from the fitted PPCA model
# Approximate the factor-loading matrix:
# W = eigenvectors * sqrt(eigenvalue - noise variance)
eigenvalues = ppca.explained_variance_
components = ppca.components_.T

latent_variances = np.maximum(eigenvalues - sigma2_est, 0)
```

```
W_est = components * np.sqrt(latent_variances)

mu_est = ppca.mean_

n_new = 10
Z_new = np.random.normal(size=(n_new, K))

# Independent isotropic Gaussian noise
epsilon_new = np.random.normal(
    0, np.sqrt(max(sigma2_est, 0)), size=(n_new, X.shape[1])
)

X_new = Z_new @ W_est.T + mu_est + epsilon_new

generated_df = pd.DataFrame(X_new, columns=columns)
print("\nTen synthetic sensor samples:")
display(generated_df.round(3))

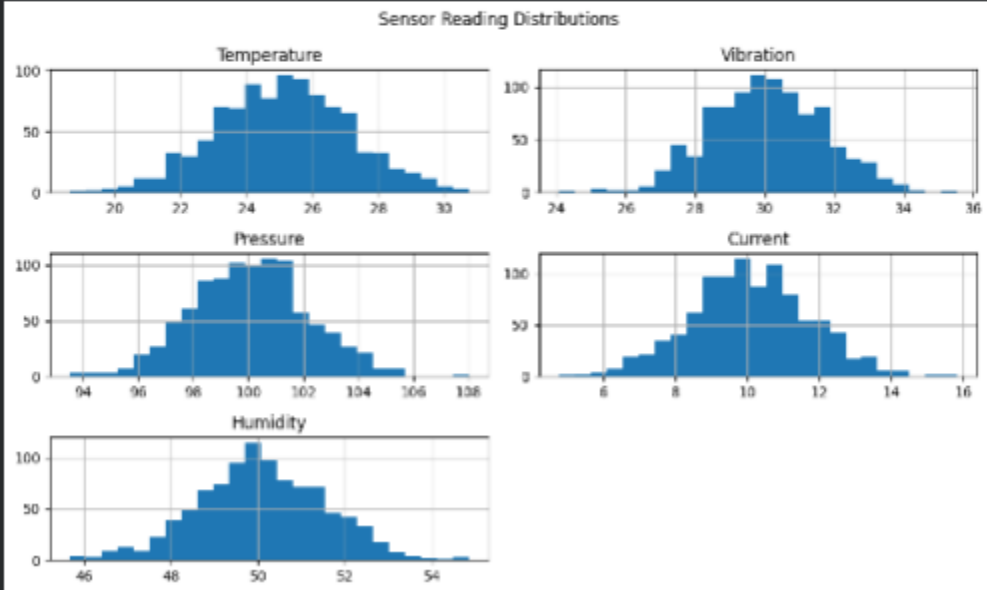
# Compare sensor means
mean_comparison = pd.DataFrame({
    "Observed Mean": X.mean(axis=0),
    "Generated Mean": X_new.mean(axis=0)
}, index=columns)

print("\nSensor mean comparison:")
display(mean_comparison.round(3))
```

Output:

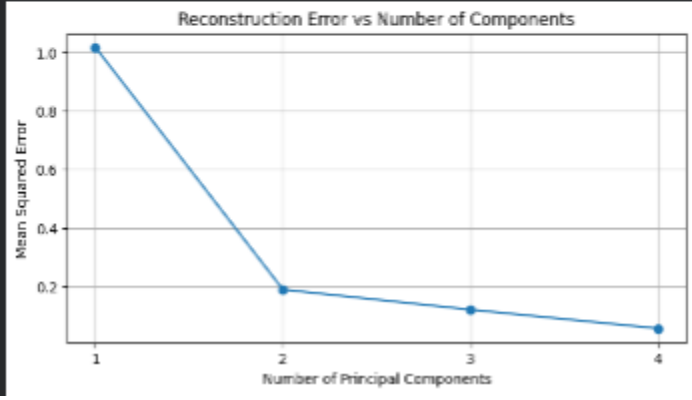
```
Missing values per sensor:
Temperature     31
Vibration       32
Pressure        40
Current         25
Humidity        25
dtype: int64

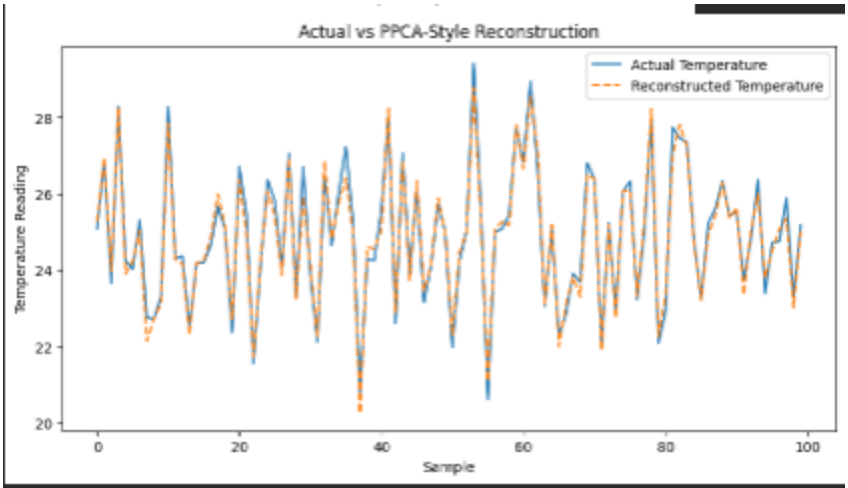
Prepared data shape: (1000, 5)
```




```
Estimated PPCA noise variance (sigma^2): 0.316363
Explained variance by each retained component:
[10.3415  4.1361]
Explained variance ratio:
[0.6704 0.2681]
Total retained variance (%): 93.85
```

	Components	Explained Variance (%)	Reconstruction MSE
0	1	67.036638	1.016015
1	2	93.847745	0.189628
2	3	96.071274	0.121093
3	4	98.149242	0.057045





Ten synthetic sensor samples:

	Temperature	Vibration	Pressure	Current	Humidity
0	28.013	32.593	101.691	12.343	52.744
1	24.643	29.417	98.086	7.926	49.034
2	26.093	31.059	102.566	11.389	50.786
3	26.720	31.744	101.330	11.397	51.487
4	21.968	26.764	96.524	6.735	46.852
5	24.727	29.171	99.545	9.459	48.649
6	27.126	31.969	101.428	12.115	51.475
7	26.495	32.335	103.078	12.116	52.621
8	26.034	32.026	99.652	9.800	50.898
9	25.133	30.592	98.606	9.110	49.354

Sensor mean comparison:

	Observed Mean	Generated Mean
Temperature	25.087	25.795
Vibration	30.067	30.767
Pressure	100.077	100.251
Current	10.075	10.239
Humidity	50.075	50.390